

Refund Execution

A buyer whose verified payment ended as `refund_due` now gets their money back **exactly once**, with evidence. This is what was built, what was run, and what the runs actually said.

Branch `claude/create-app-repository-1cxsih`

Approved base `4342c06`

Code state `3fe233d`

Files changed **36**

Unit tests **429 passed**

Browser tests **31 passed**

Mutations **24 / 26 caught**

THE CORRECTION YOU REQUIRED, AND WHERE IT LIVES

The approved design classified an HTTP 5xx as a definite failure. You corrected that before implementation, and the corrected rule is now the axis the whole phase turns on: **an HTTP status never decides whether the financial operation happened**. The classifier asks only how far the request got. A 5xx received *after* transmission is `in_doubt` , and `in_doubt` is never retried.

It is enforced in one place — `payments/refund-outcome.ts` — proven by six adversarial tests and by mutations R1, R2 and R3, each of which makes a test fail. Section 10 shows the output.

1

The code state this describes

BRANCH	claude/create-app-repository-1cxsih
APPROVED BASE	4342c06 — the Phase 2 financial gate you approved
IMPLEMENTATION COMMITS	cab8845 — Phase 3: refund execution 2f25fa5 — Freeze the refund money snapshot in the database, not by convention 3fe233d — Close two weak refund assertions that mutation testing exposed
MERGED TO MAIN	NO and will not be without your word
SETTLEMENT STARTED	NO

ON THE COMMIT NAMED ABOVE

Every test result and mutation result in this packet was produced against the tree at 3fe233d. This document and its PDF are committed *above* that commit, so the branch tip is newer — but every commit between them touches docs/ only. Verify with `git diff --stat 3fe233d HEAD -- .`

' :!docs ': it is empty.

Files changed, and why each one moved

Thirty-six files: 6 365 lines added, 45 removed. The forty-five removed lines are the only pre-existing code that changed at all, and every one of them is listed below.

New — the refund domain

FILE	LINES	WHAT IT IS
core/payments/refund-outcome.ts	252	The correction. Classifies an attempt by how far the request got, never by what the response said. Holds the closed not-transmitted set and the secret screen
core/refunds/state.ts	110	Five states as data. <code>in_doubt</code> → <code>in_flight</code> is absent from the table
core/refunds/record.ts	305	Recording a debt inside the confirming transaction; reading the queue
core/refunds/execute.ts	969	Claim → send → settle, in that order. Resolution by status query and by webhook. The queue processor
core/refunds/reconcile-manual.ts	342	Propose and confirm, under dual control where dual control is possible
core/refunds/policy.ts	99	Retry policy, clamped so configuration can only ever be gentler
core/notifications/index.ts	125	The outbox. Writes rows; sends nothing, and has no function that could claim to
core/refunds/refunds.test.ts	1 503	80 tests across 14 sections
db/migrations/0012_refund_execution.sql	445	Three tables, three triggers , twelve CHECKs, three permissions, four settings
db/schema/refunds.ts	243	The same in Drizzle
web: 1 page + 1 client component + 5 routes	639	The operator queue and its endpoints
web/e2e/refunds.spec.ts	425	7 browser tests against a production build

Changed — every one of the 45 removed lines

FILE	+ / -	THE CHANGE
core/orders/payment.ts	+89 / -15	Two things. A refund event is routed to the refund domain after the shared signature check; and reaching <code>refund_due</code> calls <code>recordRefundOwed</code> in the same transaction . The 15 removed lines are the rename of <code>event</code> to <code>chargeEvent</code> on the charge path, forced by the verified-event union
core/payments/provider.ts	+119 / -2	<code>VerifiedEvent</code> becomes a discriminated union; <code>RefundResult</code> gains <code>notExecutedEvidence</code> ; <code>getRefund</code> and <code>supportsRefundQuery</code> added
core/payments/mock.ts	+316 / -7	Twelve deterministic refund scenarios, a refund-shaped event builder, and truth indexed by our idempotency key
core/payments/ipaymoney.ts	+19 / -1	<code>getRefund</code> declared and throwing. Still non-operational
core/ledger/commerce-reconcile.ts	+115 / -3	Ten refund detectors; a <code>refundsChecked</code> count
core/authz/permissions.ts	+19 / -2	Three permissions; finance gains all three, support gains read only
core/scripts/reconcile-check.ts	+153 / -8	A second detector probe, for refunds; both probes moved into a transaction that is always rolled back
core/test/harness.ts	+26 / -0	Three tables truncated; the refund policy restored between tests
AGENTS.md	+31 / -4	The four Phase 3 rules that are not negotiable, written where the next person reads them
db/schema/commerce.ts	+13 / -0	<code>payment_events.subject</code> and <code>.refund_id</code>
i18n fr / en	+34 / -2	16 keys each, both catalogues in step

WHAT WAS NOT TOUCHED

No file under `auth/`, `consent/`, `content/`, `ratelimit/`, `ledger/flows.ts`, `ledger/post.ts`, `money/` or `commissions/` changed. Checkout and the late-payment rules are unchanged except for the one approved insertion. No existing test was weakened, deleted or skipped: the suite went from 349 to 429 passing with none removed.

Migration 0012, and what the database refuses on its own

The point of this section is that most of the safety is *not* in application code. Twelve CHECK constraints, five unique indexes and one trigger enforce it, so a future code path — or a direct `UPDATE` — fails loudly rather than quietly.

Tables

TABLE	HOLDS
refunds	One row per owed refund. Frozen amount and currency, status, attempt count, the provider reference, how the outcome was established, the evidence, who authorised it, and any standing manual proposal
refund_attempts	Append-only. One row per request, written before the request goes out, closed once
notification_outbox	Things a person should be told. Nothing sends from it
payment_events	Gains <code>subject</code> and <code>refund_id</code> . Refund webhooks reuse the existing <code>UNIQUE (provider, provider_event_id)</code> replay index rather than getting a second, weaker door

The twelve CHECK constraints on `refunds`

CONSTRAINT	REFUSES
<code>refunds_succeeded_has_evidence</code>	A refund claiming money went back with no resolution source, no evidence, no provider reference and no timestamp. This is why there is no "mark as refunded" button — one could not work
<code>refunds_failed_has_evidence</code>	A refund claiming money definitely did NOT go back, with nothing behind the claim. The other half of the rule, and the one that keeps an unknown from wearing a failure's clothes
<code>refunds_two_person_is_two_people</code>	A <code>two_person</code> reconciliation where the two ids are equal — a self-confirmation recorded as dual control
<code>refunds_proposed_success_has_ref</code>	Proposing a success without naming the provider's reference
<code>refunds_proposal_is_complete</code>	A proposal missing its evidence, its timestamp or its proposer
<code>refunds_confirmer_needs_a_proposer</code>	A confirmation of a proposal that does not exist
<code>refunds_status_valid · refunds_resolved_by_valid · refunds_reconciliation_mode_valid · refunds_proposed_outcome_valid</code>	Any value outside the enumerations
<code>refunds_amount_positive</code>	A zero or negative refund
<code>refunds_reconcilers_only_when_manual</code>	Reconcilers named on a refund nobody reconciled

Indexes and the trigger

OBJECT	GUARANTEE
refunds_payment_key	UNIQUE (payment_id). One payment, one refund, for all time — the outermost layer of the double-refund defence
refunds_order_key	UNIQUE (order_id)
refunds_provider_ref_key	Partial UNIQUE — one refund per provider reference
refund_attempts_no_key	UNIQUE (refund_id, attempt_no)
refund_attempts_idempotency_key	UNIQUE. The derived key cannot repeat
notification_outbox_event_key	UNIQUE (kind, subject_type, subject_id) — one message per event per subject
refund_attempts_append_only	Trigger. DELETE is refused outright; UPDATE is refused once finished_at is set, and refused always for refund_id , attempt_no , idempotency_key and started_at . An attempt may be closed, never rewritten
refunds_snapshot_frozen	Trigger. Refuses any UPDATE that changes amount_minor , currency , payment_id , order_id or provider . How much goes back and to whose charge is a fact, not a lifecycle — the same reason ledger entries and fee snapshots are immutable
refunds_no_delete	Trigger. A refund is resolved, never erased

THE TRIGGER CAUGHT ITS FIRST TEST

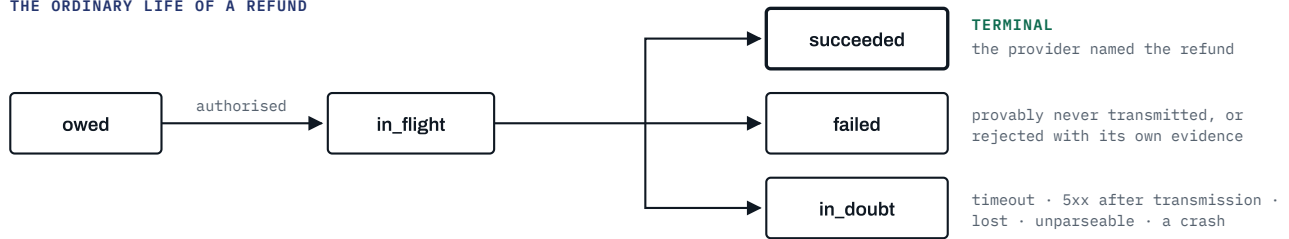
A test tried to back-date started_at to simulate a stuck request. The trigger refused it — correctly, because a re-identifiable attempt row is not evidence. The test now asserts that refusal and moves the threshold instead. That is the trigger doing its job before any production code existed.

Reference data the migration writes

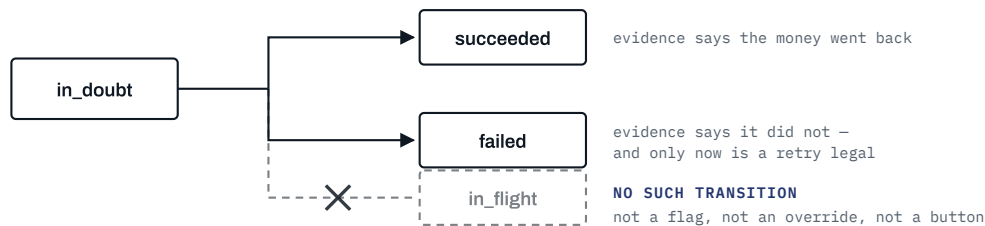
KEY	VALUE	CLAMP DIRECTION
refund.max_attempts	3	Down. Configuration may permit fewer, never more
refund.retry_backoff_seconds	300	Up. May wait longer, never less
refund.stuck_in_flight_seconds	900	Up. May be more patient before declaring unknown
refund.queue_batch_size	20	Down
refund.read · refund.execute · refund.reconcile_manual	Three permissions. finance holds all three; support holds refund.read only	

The state machine, and the one transition that is absent

THE ORDINARY LIFE OF A REFUND



AND WHAT AN UNKNOWN OUTCOME MAY DO



Seven transitions, and the eighth is deliberately missing. An unknown outcome can be *resolved* – to `succeeded` when evidence says the money went back, to `failed` when evidence says it did not – and only a resolution to `failed` makes a retry legal, one state later. There is no path out of `in_doubt` that sends a second request without first learning something.

The table is asserted directly, so a widened machine fails a test rather than a review:

```

it("permits exactly the transitions the design approved", () => {
  // every from × to pair, enumerated
  expect([...legal].sort()).toEqual([
    "failed->in_flight", "in_doubt->failed", "in_doubt->succeeded",
    "in_flight->failed", "in_flight->in_doubt", "in_flight->succeeded",
    "owed->in_flight",
  ]);
});

```


TRANSITION	WHAT CAUSES IT	ENFORCED BY
owed → in_flight	A finance user with <code>refund.execute</code> and step-up elevation	State table, guarded UPDATE, row lock
in_flight → succeeded	The provider answered, and named the refund	Guarded UPDATE + <code>refunds_succeeded_has_evidence</code>
in_flight → failed	<code>not_transmitted</code> , or a rejection carrying <code>notExecutedEvidence</code>	Guarded UPDATE + <code>refunds_failed_has_evidence</code>
in_flight → in_doubt	Everything else, plus the sweeper recovering a crash	Guarded UPDATE
failed → in_flight	A retry, inside the attempt ceiling and past the backoff	State table + policy clamps
in_doubt → succeeded	Status query, signed webhook, or two-person reconciliation	State table + CHECK
in_doubt → failed	The same three, saying no refund exists	State table + CHECK
in_doubt → in_flight	DOES NOT EXIST Attempting it throws with the reason spelled out, and the guarded UPDATE would refuse it a second time even if the check were deleted	

`refund_due` remains a **terminal order state**, as decided. A browser test asserts the order is still `refund_due` after the refund succeeds: the order records the debt, the refund records what happened, and the two cannot disagree because only one of them is tracking a lifecycle.

The provider boundary — where the correction is enforced

One file decides `failed` versus `in_doubt`, and it never reads an HTTP status to do it. Three stages, and only the first can produce a plain failure:

STAGE	MEANS	BECOMES
<code>not_transmitted</code>	We can prove the request never reached the provider	<code>FAILED</code> retryable
<code>transmitted</code>	The bytes left. What the provider did with them is a separate question	<code>IN_DOUBT</code> unless the answer carries evidence
<code>unknown</code>	We cannot establish whether they left	<code>IN_DOUBT</code>

The not-transmitted set is short, closed, and asserted

ON THE LIST	WHY IT IS PROOF
<code>ENOTFOUND</code> · <code>EAI_AGAIN</code>	DNS never resolved a host to connect to
<code>ECONNREFUSED</code> · <code>EHOSTUNREACH</code> · <code>ENETUNREACH</code> · <code>EADDRNOTAVAIL</code>	The transport refused, or could not reach the host
TLS handshake failures (5 codes)	Failed before any application bytes
<code>ERR_INVALID_URL</code>	We never built a request

DELIBERATELY ABSENT	WHY IT IS NOT PROOF
<code>ECONNRESET</code>	The peer reset — which can happen after the request was fully sent, and even after it was processed
<code>ETIMEDOUT</code>	A timeout says only that no answer came back
<code>EPIPE</code>	The socket closed mid-write; some bytes may have gone
<code>ABORT_ERR</code>	We gave up. The provider did not necessarily
<code>UND_ERR_*</code>	undici's own timeouts, all post-connection
every HTTP status	A response existing at all proves the request arrived. A 500 says the provider's web tier had a problem, and nothing about the refund it may already have accepted

```

it("holds the classification rules directly, without a database", () => {
  expect(stageOfTransportFailure({ code: "ENOTFOUND" })).toBe("not_transmitted");

  // each of these could have happened AFTER the bytes went out
  for (const code of ["ECONNRESET", "ETIMEDOUT", "EPIPE", "ABORT_ERR", "UND_ERR_HEADERS_TIMEOUT"]) {
    expect(stageOfTransportFailure({ code })).toBe("unknown");
  }
  expect(classifyRefundFailure(new RefundTransportError("500", "transmitted", 500)).kind)
    .toBe("in_doubt");
});

```

An answer must also be credible, not merely present

PROVIDER SAYS	RESULT	WHY
succeeded with a reference	SUCCEEDED	There is something reconciliation can check later
succeeded with no reference	IN_DOUBT	"It worked" with no name for what worked is not checkable
failed with notExecutedEvidence	FAILED	The provider's own statement that no money moved
failed with nothing behind it	IN_DOUBT	Indistinguishable from an accepted refund reported badly. This is all an HTTP status could ever justify, and it is not enough
pending	stays in_flight	Not a resolution. A webhook or a query ends it

The interface changes

ADDITION	PURPOSE
RefundResult.notExecutedEvidence	What makes failed mean failed
getRefund?(query)	Optional. Queryable by the provider's reference or by our idempotency key — after a timeout the key is the only handle we hold
supportsRefundQuery(provider)	Callers branch rather than assume. A provider without it makes every ambiguity a manual process, reported by name
VerifiedEvent as a union	subject: "charge" "refund", required on both. A refund event cannot be read as a charge event by code that forgot to check

NO DESTINATION, ANYWHERE

RefundInput is { providerRef, amount, reason, idempotencyKey }. There is no beneficiary field, no account field and no destination field on the interface, so no API request, server action or test is in a position to redirect a refund. The money goes back the way it came because that is the only thing the interface can express.

Idempotency — four layers, none of which trusts the others

#	LAYER	COVERS THE CASE THE OTHERS DO NOT
1	UNIQUE (payment_id)	Whatever races and whatever crashes, a second <code>refunds</code> row for one payment cannot exist. Not a read-then-write — the index is the guarantee
2	SELECT ... FOR UPDATE	Two operators pressing at the same instant serialise instead of both reading <code>owed</code>
3	Guarded transition	Every UPDATE names the state it expects, so a caller that got past the lock with a stale read matches no rows and changes nothing
4	Attempt row committed BEFORE the call	A crash between the commit and the response leaves an open attempt — evidence a request may be out there, rather than silence

On top of all four, the absolute rule: `in_doubt` is never re-sent, only resolved.

The derived key

```
/** Derived, never supplied. A caller has no say in what the provider is told. */
export function refundIdempotencyKey(refundId: string, attemptNo: number): string {
  return `refund:${refundId}:${attemptNo}`;
}
```

No route, action or test supplies a key. Two attempts on one refund get distinct keys — which is correct, because a retry only happens after the previous attempt was *proven* not to have reached the provider, and reusing a key there would make a provider that cached the failure return it forever.

What was actually run

TEST	RESULT
A second refund row for the same payment is refused by the index	PASS
Two attempts get keys <code>...:1</code> and <code>...:2</code> ; a duplicate key is refused	PASS
A finished attempt cannot be rewritten, and no attempt can be deleted	PASS
A replayed refund webhook applies once; one <code>payment_events</code> row exists	PASS
Four racing executions perform exactly one refund at the provider	PASS

7 Concurrency — against a lock actually held on a second connection

WHY NOT PROMISE.ALL

`Promise.all` cannot reliably interleave two database transactions, so a test built on it passes whether or not the lock exists. This test *holds the row*: a separate pooled connection takes `for update`, an execution arrives and must block, the holder changes the refund underneath it and commits, and only then does the waiter get to look. What it must find is the state the holder left — and refuse to act on the state it read before waiting.

```
const holder = await getPool().connect();
await holder.query("begin");
await holder.query("select id from refunds where id = $1 for update", [refundId]);

const attempt = executeRefund(db, finance, provider, refundId); // blocks
await new Promise((r) => setTimeout(r, 250)); // reach the lock

// underneath the waiting execution, the refund is claimed and sent
await holder.query("update refunds set status = 'in_flight', attempt_count = 1 where id = $1", [refundId]);
await holder.query("insert into refund_attempts (...) values (... , 1, $2)", [refundId, key]);
await holder.query("commit");

expect((await Promise.allSettled([attempt]))[0].status).toBe("rejected");
expect(provider.refundsPerformed()).toBe(0); // nothing was sent
expect(attemptCount).toBe(1); // exactly one attempt exists
```

SCENARIO	ASSERTION	RESULT
Held lock, refund claimed underneath	The waiter refuses; zero refunds performed; one attempt row	PASS
Four simultaneous executions	Exactly one refund performed; refund ends <code>succeeded</code> ; reconciliation clean	PASS
An operator and the scheduler colliding on a retryable refund	Exactly one refund performed; reconciliation clean	PASS
Mutation R7 removes <code>for update of r</code>	Tests fail	CAUGHT
Mutation R8 removes the guarded <code>where status = ...</code>	Tests fail	CAUGHT

The crash window — observed from inside the provider call

The claim is an ordering: the attempt row is **committed before the provider is called**. A test that asserted this after the fact would prove nothing, so the test observes it from *inside* the call, on a separate connection, at the exact instant a crash would occur.

```
const crashing = new MockPaymentProvider();
Object.assign(crashing, {
  refund: async () => {
    // A SEPARATE connection: only committed data is visible here, which is
    // exactly what would survive if the process died on the next line.
    const client = await getPool().connect();
    attemptsVisibleDuringCall = (await client.query(
      "select count(*)::int as n from refund_attempts where refund_id = $1", [refundId])).rows[0].n;
    refundStatusDuringCall = (await client.query(
      "select status from refunds where id = $1", [refundId])).rows[0].status;
    client.release();
    throw new RefundTransportError("process died here", "unknown");
  },
});

await executeRefund(db, finance, crashing, refundId);

expect(attemptsVisibleDuringCall).toBe(1);          // durable before the call
expect(refundStatusDuringCall).toBe("in_flight");
```

SCENARIO	ASSERTION	RESULT
Observed from inside the provider call	One attempt row committed; refund already <code>in_flight</code>	PASS
Provider never returns (the process is killed mid-flight)	Refund stays <code>in_flight</code> ; the attempt has no <code>finished_at</code> ; a second execution is refused	PASS
The sweeper reaches a crashed request	It becomes <code>in_doubt</code> — never <code>failed</code>	PASS
Mutation R5 writes the attempt row 400 ms <i>after</i> the call	Tests fail	CAUGHT
Mutation R15 makes the sweeper write <code>failed</code>	Tests fail	CAUGHT

WHY THE SWEEP LANDS WHERE IT DOES

Sweeping to `in_doubt` says "we stopped waiting", not "it failed". It unlocks nothing and permits no retry — it only removes the claim that a response is still coming. A sweep to `failed` would turn every crashed process into a duplicate refund, which is what mutation R15 demonstrates.

9 Reconciliation — ten detectors, and a gate that proves it still detects

DETECTOR	CATCHES	TESTED
refund_due_without_refund_record	A debt nothing is tracking — the queue starving silently	YES
refund_without_refund_due_order	A refund whose order is not <code>refund_due</code>	—
refund_of_unsucceeded_payment	Money going back out that never came in	YES
refund_amount_mismatch	The refund and its payment disagreeing about the money. The refund's side is now frozen by a trigger, so only the payment can drift — and the detector notices from whichever side it came	YES
multiple_refunds_for_payment	The buyer refunded twice. Unreachable today; the detector notices if the index is dropped	—
refund_succeeded_without_evidence	A success claim with no source, evidence or reference	—
refund_failed_without_evidence	An unknown wearing a failure's clothes — and therefore retryable	—
stuck_in_flight	A request out for over an hour that the sweeper should have handled	YES
refund_attempt_never_finished	Evidence and state disagreeing, which makes the evidence useless	YES
manual_reconciliation_single_operator	The weakest control in the system, counted rather than invisible	YES

Six are asserted by name in the suite. The four marked "—" describe states a CHECK constraint or a unique index already makes unreachable; they exist so that dropping the constraint is noticed rather than silent, which is exactly the change that looks harmless in a diff.

The CI gate proves the detector, not just the data

Ten checks over a table that is empty on most CI runs would report "clean" and mean nothing. So the gate builds a consistent refund, requires silence, breaks the frozen amount, requires the detector to speak, and rolls the whole thing back. Actual output:

```
"commerce": {
  "ordersChecked": 0,
  "paymentsChecked": 0,
  "refundsChecked": 0,
  "anomalies": [],
  "detectorProof": { "silentWhenConsistent": true, "detectedWhenBroken": "succeeded_payment_or",
  "refundDetectorProof": { "silentWhenConsistent": true, "detectedWhenBroken": "refund_amount_mismatch"
}
```

Ledger reconciliation clean over a real capture → settlement → payout cycle.

The counts read zero because **the probes run inside a transaction that is always rolled back**. They plant rows that are deliberately wrong, so they must leave nothing behind — and deleting afterwards turned out to be the wrong instrument: the refund probe could not delete its own refund, correctly, because a refund is resolved and never erased. A cleanup that has to disable a trigger is a cleanup that could disable the wrong one. A rollback removes everything without asking any table's permission, and removes it even if the probe throws halfway.

Each probe also asserts silence about *its own rows* rather than about the whole database. Asserting a globally empty report made the probe a hostage to whatever the suite left behind — one unrelated anomaly and a working detector would report that it cannot detect, which is the opposite of the truth.

The gate exits non-zero if either probe stops behaving — a reconciliation that cannot report a fault is not a gate. Mutations **R22**, **R23** and **R24** each break one detector and are each caught.

Tests, and the mutations that prove they are load-bearing

80 new unit tests across 14 sections, plus 7 browser tests. The suite went from 349 to 429 passing with nothing removed, weakened or skipped.

SECTION	TESTS	ESTABLISHES
1 · the state machine is the safety argument	4	Seven transitions, enumerated; <code>in_doubt</code> → <code>in_flight</code> absent; <code>succeeded</code> the only terminal state
2 · a <code>refund_due</code> order records a debt	5	Created in the confirming transaction; amount frozen from the payment; a second row refused; the order stays terminal
3 · the ordinary success path	4	Executes with evidence; one attempt with a derived key; zero ledger entries ; one notification
4 · UNKNOWN MUST NEVER BECOME FAILED	12	Timeout, 5xx, lost connection, unparseable, DNS, refused, documented rejection, bare failure, success with no reference — plus the classifier held directly, and the secret screen
5 · an <code>in_doubt</code> refund is never retried	5	Direct retry refused; the queue does not retry it either; resolution to the truth in both directions; a provider with no query leaves it alone
6 · idempotency and the crash window	6	The attempt row observed from inside the call; an abandoned request; the open-attempt guard, isolated ; the sweep; distinct keys; the append-only trigger
7 · concurrency, against a real held row lock	3	A held lock on a second connection; four racing executions; operator versus scheduler
8 · retries, bounded and clamped	5	The ceiling; the backoff; configuration that tries to be <i>more</i> aggressive; configuration that is gentler; four malformed values and a missing row
9 · the scheduler continues authority	4	Never starts an unauthorised refund; retries an authorised one; refuses a planted <code>failed</code> with no requester; sweeps and resolves in one pass
10 · authorization and the operator's hands	9	Member refused; support reads and cannot execute; elevation required; the raw <code>UPDATE</code> to <code>succeeded</code> refused by PostgreSQL; the frozen snapshot and the no-delete triggers ; a hostile status filter ; unsucceeded payment; drifted amount
11 · manual reconciliation, under dual control	8	Two people; one person and the flag; missing reference; missing evidence; no proposal; a superseded proposal; an already-resolved refund; the audit trail
12 · refund webhooks go through the same door	6	Forged signature; resolution from a signed event; replay applied once; amount mismatch; bare failure ignored; a refund event cannot move a charge
13 · reconciliation reports the impossible	7	Silent when healthy; six detectors asserted by name
14 · the audit trail, and none of it leaks	2	The whole life of a refund, in order; an unknown audited as an unknown

The mutation matrix

Each mutation is applied to the source, the suite is run, and the source is restored. A mutation that no test notices is a guarantee nobody is actually checking. The harness asserts the pattern was found before

applying, refuses to run against an already-mutated tree, and restores from a pristine snapshot rather than from whatever it happened to read.

#	MUTATION	RESULT	FIRST TEST TO FAIL
1	Classify every transport failure as <code>failed</code> , whatever its stage	CAUGHT 23 failing	UNKNOWN MUST NEVER BECOME FAILED > classifies a timeout after transmission as <code>in_doubt</code> , never failed
2	Add <code>ECONNRESET</code> , <code>ETIMEDOUT</code> , <code>EPIPE</code> , <code>ABORT_ERR</code> and <code>UND_ERR_*</code> to the proven-not-transmitted set	CAUGHT 2 failing	UNKNOWN MUST NEVER BECOME FAILED > classifies a connection lost after transmission as <code>in_doubt</code>
3	Believe a provider's <code>failed</code> that carries no evidence	CAUGHT 2 failing	UNKNOWN MUST NEVER BECOME FAILED > REFUSES a bare failure with no evidence, and calls it unknown
4	Permit <code>in_doubt → in_flight</code> in the state table	CAUGHT 4 failing	the state machine is the safety argument > forbids <code>in_doubt → in_flight</code> , and says why
5	Write the attempt row 400 ms <i>after</i> the provider call instead of before	CAUGHT 42 failing	the ordinary success path > executes, resolves and records evidence
6	Drop the refusal to dispatch while an attempt is still open	CAUGHT 1 failing	idempotency and the crash window > refuses to dispatch while an attempt is open, even from a retryable state
7	Remove <code>for update of r</code> from the claim	NOT ALONE	—
8	Drop <code>where status = ...</code> from the claim's UPDATE	NOT ALONE	—
9	Make the idempotency key ignore the attempt number, so two attempts collide	CAUGHT 5 failing	idempotency and the crash window > refuses to dispatch while an attempt is open, even from a retryable state
10	Stop comparing the frozen amount to the payment before dispatch	CAUGHT 1 failing	authorization, elevation and the operator's hands > refuses to send an amount the payment does not corroborate 24
11	Stop requiring the payment to have succeeded	CAUGHT 1 failing	authorization, elevation and the operator's hands > refuses to refund a payment that did not succeed
12	Remove <code>authorize()</code> and <code>requireElevation()</code> from execution	CAUGHT 3 failing	authorization, elevation and the operator's hands > refuses an ordinary member
13	Keep the permission check, drop the step-up elevation	CAUGHT 1 failing	authorization, elevation and the operator's hands > requires step-up elevation even from finance
14	Let the scheduler pick up <code>owed</code> refunds nobody authorised	CAUGHT 2 failing	the scheduler continues authority, it never creates it > never starts a refund nobody authorised
15	Make the stuck sweeper write <code>failed</code> instead of <code>in_doubt</code>	CAUGHT 1 failing	idempotency and the crash window > sweeps a crashed request to <code>in_doubt</code> , never to failed
16	Remove the downward clamp on <code>refund.max_attempts</code>	CAUGHT 1 failing	retries, bounded and clamped > clamps configuration that tries to make retries MORE aggressive

#	MUTATION	RESULT	FIRST TEST TO FAIL
17	Remove the upward clamp on <code>refund.retry_backoff_seconds</code>	CAUGHT 1 failing	retries, bounded and clamped > clamps configuration that tries to make retries MORE aggressive
18	Coerce a malformed setting with <code>Number()</code> instead of rejecting it	CAUGHT 1 failing	retries, bounded and clamped > falls back to the reviewed policy on a malformed setting
19	Let the proposer confirm their own manual reconciliation	CAUGHT 1 failing	manual reconciliation, under dual control > requires a second person when a second person exists
20	Allow a manual success with no provider reference	CAUGHT 1 failing	manual reconciliation, under dual control > refuses to claim a success without an external reference
21	Stop screening secrets out of stored provider detail	CAUGHT 2 failing	UNKNOWN MUST NEVER BECOME FAILED > screens secrets out of anything it stores
22	Break <code>refund_due_without_refund_record</code>	CAUGHT 1 failing	reconciliation reports what the machine says is impossible > detects a refund_due order with no refund record 225
23	Break <code>refund_amount_mismatch</code>	CAUGHT 1 failing	reconciliation reports what the machine says is impossible > detects a frozen amount that stopped matching its pa
24	Break <code>manual_reconciliation_single_operator</code>	CAUGHT 1 failing	manual reconciliation, under dual control > permits one operator when the deployment genuinely has one, and says
25	Pass the caller's status filter straight to <code>sql.raw</code>	CAUGHT 1 failing	authorization, elevation and the operator's hands > never lets a caller's status filter reach SQL
26	Remove the state assertion, the row lock <i>and</i> the guarded UPDATE together	CAUGHT 3 failing	an in_doubt refund is never retried > refuses a direct retry, with an explanation

NO MUTATION SURVIVES THAT SHOULD NOT

24 of 26 mutations fail at least one test. The remaining 2 are the redundant serialisers explained below: each is invisible alone because two other layers cover it, and the combined mutation that removes all three is caught. Nothing in the list is a guarantee the suite merely asserts in prose.

What the first run found, and why that matters more than the second

The matrix was run twice. **The first run had four survivors**, and reporting only the second would hide the most useful thing this discipline produced.

SURVIVOR	WHAT IT EXPOSED	NOW
R6	A real gap. The open-attempt guard was never exercised. The test that claimed to cover it drove the refund to <code>in_flight</code> and asserted a rejection matching <code>/still open not possible cannot go/</code> — and <code>in_flight</code> is already refused by the state machine, whose message reads "cannot go from <code>in_flight</code> to <code>in_flight</code> ". The regex matched <i>that</i> , so deleting the guard entirely changed nothing and the test still passed	The old assertion now names the state machine's message specifically, and a new test isolates the guard from a <code>failed</code> refund — a state a retry may legitimately leave from, so only an open attempt can stand in the way. CAUGHT
R18	A real gap. The malformed-setting test passed against a coercing loader by accident: <code>Number()</code> turns <code>"null"</code> , <code>"3"</code> and <code>"lots"</code> into 0, 3 and NaN, and every one of those lands back on the reviewed default of 3. A loader that accepted strings as configuration looked identical to one that rejected them	The list now includes the string <code>"1"</code> , which is the value that separates them, plus <code>"9"</code> , <code>true</code> and <code>{}</code> . CAUGHT
R7 · R8	Not a gap — the design. The row lock, the guarded UPDATE and the in-code state assertion each cover what the others cover, so removing any one alone is invisible. No single-mutation test can distinguish three redundant layers, and a test written to force one would be testing the test	A combined mutation R26 removes all three together and IS CAUGHT — the honest evidence that the set is load-bearing while no single member is individually necessary. R7 and R8 stay in the matrix, marked NOT ALONE , rather than being quietly deleted

THE POINT OF THE EXERCISE

Two of the four survivors were assertions that *passed for the wrong reason* — the most expensive kind of test, because it reads as coverage and provides none. Neither would have been found by reading the suite, and both were found by deleting a line and watching nothing happen. The second run's clean sweep is the weaker result; this table is the stronger one.

A BUG THE HARNESS ITSELF FOUND

An early mutation run was killed by a timeout before it could restore the source, and the *next* run then captured the mutated file as its own backup and restored that — quietly making a mutation into the code. Two mutations survived that way for several minutes and were caught because a test that had been passing started failing consistently. The harness now snapshots every touched file into a pristine directory once, verifies the tree is clean before applying anything, and restores from the snapshot. The whole matrix above was run after that fix, from a verified-clean tree.

Three capabilities, not one

PERMISSION	ALLOWS	HELD BY
<code>refund.read</code>	See the queue and the attempt history	<code>finance</code> , <code>support</code>
<code>refund.execute</code>	Send a refund; ask the provider about one	<code>finance</code>
<code>refund.reconcile_manual</code>	Assert a fact about money that no API confirmed	<code>finance</code>

Support can answer "where is my refund?" and can do nothing else. Reading is not touching, and the split is why that is a property rather than a promise.

Two gates on execution, answering different questions

GATE	ASKS
<code>authorize(db, actor, "refund.execute")</code>	May this actor <i>ever</i> execute refunds?
<code>requireElevation(actor, "refund.execute")</code>	Did they re-verify recently — are they present right now, rather than a stolen cookie from last Tuesday?

Both live in the **domain function**, not in the route handler. A check in a route protects one route; a check in the domain protects every caller, including the queue processor and anything added later.

Direct API manipulation, proved in a browser

ATTEMPT	RESULT
An ordinary buyer <code>GET /api/v1/admin/refunds</code>	<code>403</code>
An ordinary buyer <code>POST .../execute</code>	<code>403</code> — refund stays <code>owed</code>
An ordinary buyer <code>POST .../process</code> (the scheduler door)	<code>403</code>
Support, who <i>can</i> read the queue, <code>POST .../execute</code>	<code>403</code> — refund stays <code>owed</code>
Finance without a fresh verification, <code>POST .../execute</code>	<code>403 AUTH.ELEVATION_REQUIRED</code> — refund stays <code>owed</code>

The support case is the important one: the page renders no send button for that operator, and the route refuses them anyway. UI absence is a courtesy; the domain is the control.

The security risks named in the design, and what answers each now

RISK	WHAT ANSWERS IT	EVIDENCE
An operator fabricates a success	No "mark as refunded" exists; the CHECK refuses a succeeded refund with no source, evidence or reference	Test asserts the raw <code>UPDATE</code> is refused by PostgreSQL
Refund redirected to an attacker	No destination field exists on the interface	Structural — <code>RefundInput</code> has four fields, none of them a destination
Amount or currency tampering	Re-read from <code>payments</code> under the lock and compared before the call; a detector re-checks it	Test edits the row and the send is refused; mutation R10 caught
Forged or replayed refund webhook	Same signature-over-raw-bytes boundary and the same replay index as charges	Forged signature refused; replay applies once
Authorization bypass by direct API call	<code>authorize()</code> in the domain, actor scoped in SQL	Five browser assertions above; mutations R12 and R13 caught
Provider credentials leaking into records	Responses truncated to 500 chars and screened for bearer tokens, key/secret pairs and JWTs before storage	A test drives a provider that leaks <code>Bearer sk_live_TOPSECRET99</code> and asserts it appears in no attempt detail, no resolution evidence and no audit context; mutation R21 caught
SQL injection through the queue's status filter	The filter is intersected with <code>REFUND_STATES</code> before it reaches <code>sql.raw</code> . TypeScript said the values were safe, but a value arriving from JSON has no type at runtime	A test passes <code>''; drop table refunds; --</code> and asserts an empty result and an intact table; mutation R25 caught
An edited refund row sending the wrong amount	The money snapshot is frozen by a trigger , not by convention — amount, currency, payment, order and provider cannot be UPDATED at all, and the row cannot be DELETED	A test asserts three UPDATES and one DELETE are each refused by PostgreSQL
The scheduler endpoint reachable by anyone	A machine token compared in constant time , or a session actor with <code>refund.execute</code> . An unset token is not a token that matches an empty header	Browser: an ordinary buyer gets 403

Ten audit actions, and what is deliberately not in them

ACTION	RECORDED WHEN
refund.owed_recorded	A debt is created, inside the confirming transaction
refund.execution_requested	Somebody asks — logged before the claim, so refusals are on the record too
refund.attempt_started	An attempt is claimed, with its derived key
refund.succeeded · refund.failed · refund.in_doubt	An attempt settles, with its stage and source
refund.manual_reconciliation_proposed	A person states what a statement says
refund.manual_reconciliation_confirmed	A second person agrees — recording both ids, the mode, and how many people could have confirmed
refund.queue_processed	A scheduler pass, with its counts

The audit log holds the *screened* detail, never a raw provider body, and never a credential. It does hold the idempotency key — a derived, non-secret identifier, and the thread that ties an audit line to a provider request during an investigation.

Browser tests — seven, against a production build

Driven by a real browser over a real session cookie against `next start`, with the refund created the way production creates one: a real purchase, a real charge, a real HMAC-signed webhook arriving after the grace window, and the late-payment policy deciding `refund_due`. Nothing in the file inserts a refund.

TEST	WHAT IT PROVES	RESULT
The queue, and finance sending a refund	The refund appears as <code>owed</code> at 15 000 XOF; sending it reaches <code>succeeded</code> with <code>resolved_by = response</code> and a provider reference; exactly one attempt; zero ledger entries ; and <code>page.getByText(/mark as refunded/i)</code> has count 0	PASS
An ordinary buyer	No queue renders; three API routes answer 403; the refund stays <code>owed</code>	PASS
Support	Reads the queue, has no send button rendered, and is refused 403 when calling the route directly	PASS
Finance without elevation	403 <code>auth.elevation_required</code> ; the refund stays <code>owed</code>	PASS
An unknown outcome	The explainer is visible; no send button exists ; calling the route anyway returns 400 with "never retried / resolve it first"; what is offered is "ask the provider"	PASS
Manual reconciliation	Success without a reference → 400; proposing moves nothing; the proposer confirming their own proposal → 400; a second browser context with its own session confirms and the refund moves; both ids differ on the row	PASS
The outbox	One <code>refund.succeeded</code> row for the subject; zero rows with a status other than <code>pending</code>	PASS

31 passed (54.9s) — the whole browser suite: 24 pre-existing + 7 new

What was run, and exactly what it said

COMMAND	OUTPUT	
pnpm lint	5 packages · Done, 0 warnings	CLEAN
pnpm typecheck	5 packages · Done	CLEAN
pnpm test	Test Files 23 passed · Tests 429 passed 3 skipped (432)	GREEN
pnpm build	Production build, all routes compiled	GREEN
playwright test	31 passed (1.1m)	GREEN
tsx scripts/reconcile-check.ts	Both detector probes behaved; ledger clean	GREEN
26 mutations	24 caught, 2 redundant, 0 escaped	GREEN

The 3 skipped tests are the iPayMoney sandbox integration tests, skipped for the reason they have always been skipped: no sandbox credentials exist. They turn red the moment a real sandbox run regresses.

ON THE CI BADGE

Everything above was run **locally** against PostgreSQL 16, at 3fe233d, and the outputs are quoted verbatim. GitHub Actions has not yet run this commit — **CI is not verified for Phase 3**, and this packet does not claim it is. The workflow is unchanged and already runs lint, typecheck, the full suite, the reconciliation gate, the production build and the browser suite; if the run comes back red, this note becomes the finding and the packet is reissued.

14 Manual reconciliation — dual control that adapts without becoming a fiction

This is the only place a human's word becomes financial truth, so it is the most dangerous operation in the phase. Four things stand between an operator and a fabricated success:

1 An external reference is required for a success

Claiming money went back means naming what the provider calls the refund — a claim somebody else can check against the provider's own records later. The route refuses without it, and so does a CHECK constraint.

2 Step-up elevation

Holding `refund.reconcile_manual` says allowed in principle. Elevation says present right now.

3 Two people, where the deployment has two people

One proposes what a statement says; a *different* one confirms. Proposing changes nothing about the refund. Neither half moves money alone.

4 A distinct resolution source

`resolved_by = 'manual_reconciliation'` is its own value, so every refund resolved this way can be listed, counted and re-audited by somebody who was not involved.

Your decision 4, implemented as stated

You asked for B — two-person where operationally possible — with A as a temporary fallback, and said explicitly: *do not invent a second-user requirement that cannot actually be enforced.*

So the requirement counts the actual holders in SQL at the moment of confirmation:

```
const eligible = await countManualReconcilers(db); // distinct live global holders
const samePerson = proposer === actor.userId;

if (samePerson && eligible > 1) {
  throw new ManualReconciliationError("refund.second_person_required",
    "Another person who can reconcile refunds must confirm this. You proposed it, "
    + "so you may not also confirm it.");
}
const mode = samePerson ? "single_operator" : "two_person";
```

DEPLOYMENT	BEHAVIOUR	RECORDED AS
Two or more holders	The proposer cannot confirm. No flag, no override, no force parameter	two_person
Exactly one holder — Afrinext today	The same person may confirm, with elevation and the full evidence trail	single_operator

A `single_operator` resolution is **not silently identical to dual control**. It is a distinct stored value, and a reconciliation detector reports it by name on every run — so the weakest control in the system is countable rather than invisible.

THE ADAPTATION IS NOT A HOLE AN OPERATOR CAN OPEN

A finance user cannot revoke the second holder to gain self-confirmation: changing role assignments needs `admin.role.grant`, which the `finance` role deliberately does not hold. The separation of duty that already existed for withdrawals is what makes this rule enforceable.

What was run

TEST	RESULT
Two holders: the proposer is refused, a second person succeeds, <code>mode = two_person</code> , the reference lands on the row	PASS
One holder (all other finance assignments revoked): the same person may confirm, <code>mode = single_operator</code> , and reconciliation reports it	PASS
Claiming a success with no provider reference	REFUSED
A reconciliation with blank evidence	REFUSED
Confirming a proposal that does not exist	REFUSED
A new proposal clears a prior confirmation — the second person confirmed the <i>old</i> statement	PASS
Reconciling a refund the machinery already resolved	REFUSED
Both ids and the eligible count appear in the audit context	PASS
Browser: a genuinely separate session confirms; the ids differ on the row	PASS
Mutations R19 (self-confirmation) and R20 (no reference)	CAUGHT

The notification outbox — a queue with no sender, on purpose

Your decision 6 was to add the outbox now and connect no SMS provider. That is exactly what exists.

PROPERTY	HOW
Written in the same transaction as the fact it announces	A send inside the transaction blocks it on a third party; a send after it is a message lost to a crash. A row commits with the fact — either both happened or neither did
One message per event per subject	<code>UNIQUE (kind, subject_type, subject_id)</code> with <code>on conflict do nothing</code> . A retried resolution tells the buyer once
Facts, never a rendered message	The payload holds the outcome and the ids. Translation and formatting belong to whichever sender is eventually chosen, which knows the recipient's locale
Nothing can claim delivery	There is deliberately no <code>markSent</code> function. A function that moved rows to <code>sent</code> with no provider behind it would put a lie in the database, and "the buyer was notified" is exactly the lie that surfaces during a dispute

Four kinds are queued: `refund.owed` when the debt is recorded — deliberately at the moment the buyer starts wondering where their money went, not when it arrives — and `refund.succeeded`, `refund.failed`, `refund.in_doubt` on resolution.

The browser test asserts one `refund.succeeded` row for the subject and **zero** rows in any status other than `pending`. No SMS provider has been chosen for Niger and none was invented.

Everything in this section is a claim this packet does **not** make.

NOT VERIFIED	WHAT THAT MEANS
CI has not run this commit	Every result here is from a local run against PostgreSQL 16. The workflow is unchanged and covers all of it, but the GitHub Actions run for 3fe233d has not happened
No real payment provider has ever been called	Every refund in every test went to the mock. The mock proves the domain survives twelve ways a provider can fail; it proves <i>nothing</i> about iPayMoney
Nothing about iPayMoney's refund API	Not an endpoint, not an error code, not a payload, not a signature scheme. The adapter throws
No SMS was ever delivered	No provider is chosen. Rows accumulate; nothing sends
The step-up re-verification screen	Elevation is enforced and its refusal is proved in a browser, but the browser tests set <code>session.elevatedAt</code> directly — there is no re-verification UI yet to drive
Long-horizon queue behaviour	The stuck sweeper is tested by moving the threshold, not by waiting fifteen minutes. Behaviour over days, with a real backlog, is unobserved
Load and volume	The queue has never held more than a handful of rows
The secret screen against real provider output	It catches bearer tokens, key/secret pairs and JWTs. It is a net, not a proof — no screen catches every shape a credential takes
Refund of anything but a full amount	Partial refunds are not implemented, by decision 2

RISK	STATUS	WHERE IT STANDS
Double refund	FOUR LAYERS + MUTATIONS	Unique index, row lock, guarded transition, attempt-before-call, and <code>in_doubt</code> never re-sent. Every layer has a mutation that breaks it and a test that catches the break. Residual risk: a provider that ignores idempotency <i>and</i> a path we have not modelled
An unknown misread as a failure	ENFORCED IN ONE PLACE	The classifier, plus <code>refunds_failed_has_evidence</code> at the database. R1-R3 caught
A refund owed and never paid	DETECTED, NOT PREVENTED	<code>refund_due_without_refund_record</code> and <code>stuck_in_flight</code> run in CI; the queue is age-sorted. But nothing forces anyone to look at the queue . This is an operational risk, not a code one, and it does not have a code fix
The ledger does not know refunds exist	ACCEPTED, DECISION 3	Phase 2 posts no capture, so a reversal would have nothing to reverse. The <code>refunds</code> table is the only financial record until Settlement records both
Provider cannot answer status queries	UNMITIGABLE IN CODE	If iPayMoney has no refund status query (R3) or no lookup by our idempotency key (R4), every timed-out refund becomes a person reading a statement . The code supports that path; it is a staffing cost that scales with volume, and finance must be told before launch
Minor-unit or currency error	STRUCTURALLY IMPOSSIBLE	<code>bigint</code> minor units copied wholesale. Nothing in the refund path performs arithmetic on the amount, so there is nowhere for an exponent mistake to enter – and a trigger now refuses to let the copied value change at all
Refunding a payment that never succeeded	TWO CHECKS	Creation requires <code>status = 'succeeded'</code> in the same statement; dispatch re-checks under the lock; a detector re-checks after
A single operator fabricating a reconciliation	REDUCED, AND COUNTED	With one finance user, dual control cannot bind. External reference, elevation, full audit and a reconciliation detector are what remain. The mitigation that actually closes it is hiring a second finance user , and the detector is what makes the gap visible until then

STATED PLAINLY

No iPayMoney integration exists and none was attempted. Nothing about their refund endpoint, payloads, error codes or signature scheme has been invented anywhere in the codebase.

`refund()` throws. `getRefund()` is declared and throws. The adapter is non-operational and will remain so until the official contract and sandbox credentials are verified.

Ten refund facts are now documented in `docs/providers/ipaymoney/README.md`. Two are **design-changing** rather than parametric:

#	REQUIRED	WHY IT CHANGES THE DESIGN
R3	Does a refund status query exist, and what can it be queried by?	The only mechanical way an unknown is ever resolved. Without it, every ambiguous refund waits for a person
R4	Do refunds accept a caller idempotency key, and can a refund be looked up by that key afterwards?	After a timeout we hold no provider refund reference — our key is the only handle. Without lookup-by-key, R3 cannot resolve the exact case that most needs resolving
R2	Which error codes mean, in their own words, "no refund was created"	Not design-changing, but expensive if absent: with no documented list, every provider rejection becomes a manual reconciliation instead of a safe retry
R1 · R5–R10	Endpoint and enumeration; refund webhooks and their signature; whether the webhook names the charge; time limits; partial support; post-acceptance failure; sandbox controls for rejection, timeout and duplicate	Parametric — they fill in an adapter that already has a shape

THE ANSWER THAT COSTS MONEY IS "NO" TO BOTH R3 AND R4

If iPayMoney can neither be asked about a refund nor be asked about our own key, then **every refund whose request does not come back becomes a human reading an iPayMoney statement and reconciling it under dual control**. That path exists and works — it is not a gap. But it is a staffing cost that scales with transaction volume, and whoever staffs finance should learn it before launch rather than on the first timeout.

Why `getRefund` is declared-and-throwing while `createPayout` is absent: omitting a method asserts the provider may lack the capability, and `supportsRefundQuery()` returning `false` makes callers treat every ambiguity as manual *by design*. Declaring it keeps the adapter non-operational without deciding R3 on iPayMoney's behalf.

NOT STARTED, AND NOT STARTABLE

Settlement remains blocked until this implementation passes its review gate. Nothing in this commit touches `ledger/flows.ts`, `ledger/post.ts`, settlement holds or payout logic. Nothing is merged to `main`.

If you approve, the next milestone is Settlement

It is the phase that makes the ledger true about commerce, and it has to record *both* directions at once:

SCOPE	WHY IT BELONGS TO SETTLEMENT AND NOT HERE
Posting a capture when a payment is confirmed	Phase 2 deliberately posts nothing. Until a capture exists there is nothing for a refund to reverse — the reason decision 3 was A
Posting a refund reversal	Reads the <code>refunds</code> table this phase built. The evidence is already there and frozen
Seller entitlement and hold windows	Reads the frozen fee schedules Phase 2 already writes
Reconciling the ledger against commerce	Both halves exist separately today; Settlement is where they must agree

What I would ask you before starting it

1. Whether a capture posts on confirmation, or on the provider's own settlement report — they are different instants, and the gap is real money sitting somewhere.
2. Whether a refunded sale reverses the seller's entitlement, and what happens if it was already withdrawn.
3. Whether the provider's fees are posted, and against which account.

Two smaller things, if you would rather do them first

CANDIDATE	CASE FOR DOING IT NOW
The step-up re-verification screen	Elevation is enforced and refused correctly, but there is no UI to obtain it. A finance user cannot currently elevate through the product — only through the database. That is a real gap in an otherwise complete control
Getting the iPayMoney refund answers	R3 and R4 are design-changing. Learning them before Settlement is cheaper than learning them after

Nothing is merged. Settlement is not started. This branch is `claude/create-app-repository-1cxsih` at `3fe233d`, and it waits for your review.

Phase 3 · refund execution · implementation review · branch `claude/create-app-repository-1cxsih` ·
code state `3fe233d`